

Karadeniz Technical University – Software Engineering Department
SE2002-Programming Languages – Term Project I

Start Date: 30/03/2026 13:00

End Date: 05/05/2026 23:55

Lecturer: Dr. Çağatay Murat Yılmaz

Detailed information will be given in the lecture.

Assignments must be uploaded to KTU Distance Education Application system

(<https://uek.ktu.edu.tr/ogrenci.html>).

Mini Language Parser and Parse Tree Generator

Objective:

In this project, students are expected to develop a software tool that reads a formal grammar defined in BNF (Backus-Naur Form) from a file, checks whether each test sentence complies with the grammar rules, and produces a parse tree and JSON representation for valid sentences.

The aim of this project is to help students understand:

- formal grammars
- syntax analysis
- recursive parsing
- parse tree and JSON representation generation
- syntax error detection

Program Requirements

The program must:

- Read grammar rules from grammar.txt
- Read test sentences from sentences.txt
- Check whether each sentence is syntactically valid according to the grammar
- Produce a parse tree and JSON representation for each valid sentence
- Produce a clear error message for each invalid sentence

Input:

1. grammar.txt: Contains grammar rules written in BNF format.
2. sentences.txt: Contains test sentences (one per line) to validate against the grammar.

Output:

For each sentence, the program should produce:

If valid:

- Input sentence
- Valid
- Parse tree
- JSON representation of the parse tree

If invalid:

- Input sentence
- Invalid

- A clear error message indicating:
 - where the error occurs
 - what was expected
 - why the sentence is invalid

Remarks:

1. There will be at most one rule on each line, i.e. the RHS of a rule will not appear on more than one line. Alternative RHSs in a rule will be separated by a vertical bar on the same line. The number of rules in a row may vary according to the grammar.
2. The grammar rules must use ::= as the separator between the left-hand side and the right-hand side of each production rule.
3. The program should work for different grammars (grammar.txt) and for different query sentences (sentences.txt). It must not work only for the example given above.
4. The project can be implemented in C, Java, or Python.
5. Assignments can be done by groups of maximum three students. Presentations will be taken on the date announced. Source codes and project report will be submitted at the time of assignment submission.
6. Project Report Requirements
 - *Format:* The project report must be prepared using LaTeX and must strictly follow the "ktu_project_template".
 - *Content:* Students must submit a report including:
 - The grammar used for testing
 - Explanation of how grammar rules are read and processed
 - Parsing method or algorithm used in the program
 - Data structures used during parsing
 - Sample valid inputs with corresponding parse tree outputs
 - Sample valid inputs with JSON representations
 - Sample invalid inputs with corresponding error messages
 - Explanation of how syntax errors are detected and reported
 - Difficulties encountered during implementation (if any)
7. Source Code and Algorithm Implementation
 - Core Parsing Logic: The Recursive Descent Parser (or similar syntax analysis algorithms) that forms the foundation of the project must be coded by the students themselves.
 - Originality Score: Implementing the core parsing logic manually constitutes the basis of the originality score (up to 30 points).
 - Library Restrictions: If ready-made parsing libraries such as NLTK, Lark, PLY (Python Lex-Yacc), or similar tools are used, a significant point deduction will be applied, and a maximum of 5 points will be awarded for this section.
 - Standard Libraries: Standard libraries may be used freely for tasks outside the core algorithm, such as file I/O, JSON formatting, or basic string operations.
8. Artificial Intelligence (AI) Usage Policy
 - Permitted Use: AI tools may be used as a supportive resource for optimizing specific code segments, debugging, or generating documentation.
 - Prohibited Use: It is strictly forbidden to have the entire project or the main parsing algorithm generated by AI tools. The submitted work must reflect the student's own understanding and implementation.

- Evaluation: During the presentation and defense phase, the student's mastery of their own code and the algorithm used will be a decisive factor in grading.

Grading Rubric

Projects will be evaluated according to the following criteria:

1. Correctness of outputs produced for two valid input sentences (15 points)
2. Correctness of syntax error detection for two invalid input sentences (15 points)
3. Accuracy of parse tree generation for two valid sentences (15 points)
4. Correctness of JSON output structure for two valid sentences (10 points)
5. Originality of source code and student implementation:
 - a. Up to **5 points** if the solution relies heavily on external parsing libraries (e.g., NLTK, Lark)
 - b. Up to **30 points** if the core parsing logic (Recursive Descent, etc.) is implemented by the students
6. Submission of a properly prepared project report (0–15 points)

Examples

Example grammar1.txt (in BNF format):

```
<sentence> ::= <noun-phrase> <verb-phrase>
<noun-phrase> ::= <determiner> <noun>
<verb-phrase> ::= <verb> | <verb> <noun-phrase>
<determiner> ::= the | a
<noun> ::= cat | dog | man | telescope
<verb> ::= saw | liked | admired
```

Example sentences1.txt:

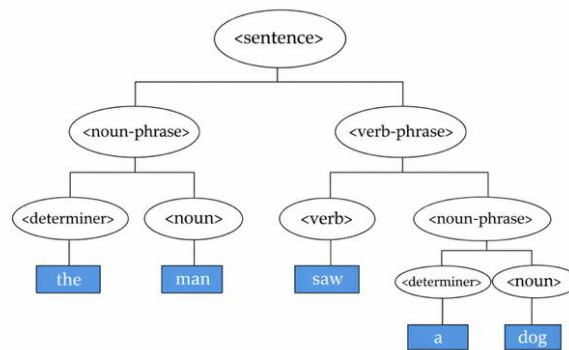
```
the man saw a dog
a telescope admired the cat
the man the dog
a man liked
liked the dog
```

Example Output:

Input: the man saw a dog

Valid

Parse tree:



JSON:

```

{
  "sentence": {
    "noun-phrase": {
      "determiner": "the",
      "noun": "man"
    },
    "verb-phrase": {
      "verb": "saw",
      "noun-phrase": {
        "determiner": "a",
        "noun": "dog"
      }
    }
  }
}

```

Example Output:

Input: liked the dog

Invalid

Error:

- Where the error occurs: at token 1 ("liked")
- What was expected: a determiner ("the" or "a") to start
- Why the sentence is invalid: the sentence begins with a verb, but grammar requires a noun phrase before the verb phrase

Example grammar2.txt (in BNF format):

```

S ::= A B
A ::= a A | ε
B ::= b B | ε

```

Example sentences2.txt:

```

ab
aaaaaaaaabbbbbbbbbb
aaaaaaa

```

```

ba
abab

```

← boş string ε